POS TECH

MACHINE LEARNING ENGINEERING

FASE 1 | AULA 03 -

DOCUMENTAÇÃO E EXEMPLOS DE USO

PDF exclusivo para Julia Soares Pereira - rm373605
jusoper@gmail.com

SUMÁRIO

O QUE VEM POR AÍ?.....3
HANDS ON 4
SAIBA MAIS.....7
MERCADO, CASES E TENDÊNCIAS 10
O QUE VOCÊ VIU NESTA AULA? 11
REFERÊNCIAS.....12

FIAP

PDF exclusivo para Julia Soares Pereira - rm373605
jusoper@gmail.com

O QUE VEM POR AÍ?

Documentar bibliotecas internas e SDKs não é perfumaria — é infraestrutura cognitiva. Em equipes grandes, o código é lido muito mais do que escrito; sem documentação, o custo de manutenção cresce de forma não linear e o risco operacional explode.

Nesta aula, você aprenderá como documentar (docstrings bem escritas), com o quê (Sphinx, MkDocs, pdoc), em que formato (reST/Markdown; estilos Google e NumPy) e por quê (qualidade, segurança, reuso e onboarding rápido). Vamos nos apoiar em guias canônicos como PEP 257 (convenções de docstrings), PEP 484 (type hints) e no framework Diátaxis para separar tutorial, how-to, referência e explicação — o mapa mental que evita “docs Frankenstein”.

Também veremos o ecossistema “docs-as-code”: Sphinx (autodoc/napoleon converte docstrings Google/NumPy em páginas), MkDocs (estático e simples), Read the Docs (hospedagem/versionamento) e ferramentas de qualidade como doctest/pytest, nbval (validação de notebooks), além de medidores de cobertura de docstring (interrogate).

Ao final, você conseguirá planejar a documentação de um SDK interno de dados, padronizar docstrings, publicar um site de docs e acoplar checagens de qualidade no CI. Assista às videoaulas para consolidar cada passo.

FIAP

PDF exclusivo para Julia Soares Pereira - rm373605
jusoper@gmail.com

HANDS ON

Nestas videoaulas, vamos criar uma minibiblioteca interna em Python chamada calcstats, padronizar docstrings em estilo NumPy, habilitar type hints (PEP 484) e gerar documentação automática com Sphinx + autodoc + napoleon. Você verá a execução do código e o site gerado, com links para fontes oficiais ao longo da explicação. Dica: assista com seu editor aberto e replique os passos em paralelo.

Nos vídeos, a estrutura é incremental: 1) escrever funções com docstrings consistentes e exemplos executáveis; 2) ligar extensões do Sphinx para extrair assinatura/tipos diretamente do código; 3) gerar páginas HTML. Reforçamos boas práticas (sumário curto, seções “Parameters/Returns/Raises/Examples”, tipos coerentes com typing) e mostramos onde o napoleon converte Google/NumPy style para reST.

Fechamos com “docs-as-code” no seu repositório: docs/ versionado, build local e integração posterior com hospedagem (ex.: Read the Docs). Depois dos vídeos, revise a sessão “Saiba Mais” para aprofundar fundamentos, métricas e trade-offs (Sphinx vs. MkDocs vs. pdoc).

```
# Linguagem: Python
# arquivo: calcstats.py
from typing import Iterable, Tuple
import math

def mean(values: Iterable[float]) -> float:
    """
    Compute the arithmetic mean.

    Parameters
    ----------
    values : Iterable[float]
        Numeric sequence.

    Returns
    -------
    float
        The arithmetic mean.

    Raises
    ------
```

PDF exclusivo para Julia Soares Pereira - rm373605
jusoper@gmail.com

```
    ValueError
        If 'values' is empty.

    Examples
    --------
    >>> mean([1, 2, 3])
    2.0
    """
    vals = list(values)
    if not vals:
        raise ValueError("Empty sequence")
    return sum(vals) / len(vals)

def mean_std(values: Iterable[float]) -> Tuple[float, float]:
    """
    Compute mean and (population) standard deviation.

    Parameters
    ----------
    values : Iterable[float]
        Numeric sequence.

    Returns
       -------
    (float, float)
        (mean, std).

    Examples
    --------
    >>> round(mean_std([1, 2, 3])[1], 6)
    0.816497
    """
    vals = list(values)
    mu = mean(vals)
    var = sum((x - mu) ** 2 for x in vals) / len(vals)
```

Código-fonte 1 – Módulo da biblioteca interna (calcstats.py)
Fonte: Elaborado pelo autor (2025)

PDF exclusivo para Julia Soares Pereira - rm373605
jusoper@gmail.com

```python
# Linguagem: Python
# arquivo: docs/conf.py
import os, sys
sys.path.insert(0, os.path.abspath('..'))  # permite importar calcstats
project = "calcstats (SDK interno)"
extensions = ["sphinx.ext.autodoc", "sphinx.ext.napoleon"]
napoleon_google_docstring = False
napoleon_numpy_docstring = True
html_theme = "alabaster"
```

Código-fonte 2 – Configuração do Sphinx (docs/conf.py)
Fonte: Elaborado pelo autor (2025)

```python
# Linguagem: Python#
# arquivo: build_docs.py
import subprocess, os
os.makedirs("docs", exist_ok=True)
# gera stub de API a partir do código
subprocess.run(["sphinx-apidoc", "-f", "-o", "docs", "."], check=True)
# compila HTML
subprocess.run(["sphinx-build", "-b", "html", "docs", "docs/_build"],
check=True)
```

Código-fonte 3 – Build programático dos docs (build_docs.py)
Fonte: Elaborado pelo autor (2025)

PDF exclusivo para Julia Soares Pereira - rm373605
jusoper@gmail.com

SAIBA MAIS

Bibliotecas internas concentram regras de negócio, invariantes e contratos que sustentam dezenas de produtos; sua documentação tem efeito de rede: uma hora investida economiza semanas em múltiplos squads.

Na prática, docstrings + site de referência + guias “how-to” reduzem mean time to understand (MTTU) e lead time to change e diminuem riscos de segurança/compliance por padronizar uso de APIs. O framework Diátaxis ajuda a organizar: Tutorial (aprendizado guiado), How-to (receitas), Referência (fatos) e Explicação (racional/teoria), evitando confundir quem lê com textos híbridos.

O ponto de partida é a PEP 257, que define como escrever docstrings (sumário curto + linha em branco + detalhes; consistência e legibilidade). Para enriquecer as assinaturas, use type hints (PEP 484) — elas melhoram a leitura, permitem static checking (mypy, pyright) e são lidas por geradores. Sphinx com autodoc importa o módulo e extrai docstrings; napoleon converte estilos Google/NumPy para reST. Para projetos de dados, o NumPy docstring standard garante seções “Parameters/Returns/Raises/Examples” uniformes (e testes via doctest).

Docstrings com exemplos executáveis (doctest) elevam a qualidade: além de ensinar, validam comportamentos em CI. O módulo doctest transforma trechos do REPL em testes; o pytest coleta doctests nativamente. Em projetos que usam notebooks como artefatos de ensino e validação, o nbval reexecuta cadernos e compara saídas, garantindo reprodutibilidade.

Geradores de site: Sphinx, MkDocs e pdoc

- **Sphinx**: poderoso, extensível, ideal para SDKs com API extensa; autodoc + napoleon tornam docstrings a fonte única da verdade. Ecossistema rico (intersphinx, sphinx-gallery para exemplos executados, MyST para Markdown).
- **MkDocs**: simples e veloz; escreve-se em Markdown com navegação via mkdocs.yml. MkDocs Material adiciona UX excelente; integra bem com Backstage/TechDocs e Read the Docs.

PDF exclusivo para Julia Soares Pereira - rm373605
jusoper@gmail.com

- **pdoc**: gera referência de API rápida diretamente das docstrings, com suporte a Google/NumPy e type hints; útil para libs menores ou para preview local.

CRITÉRIO	SPHINX	MKDOCS	PDOC
Tamanho/complexidade da API	Alto	Médio/baixo	Baixo
Auto-API a partir de docstrings	Excelente (autodoc)	Via plugins externos	Sim (nativo)
Curva de aprendizado	Maior	Menor	Mínima
Integração com TechDocs (Backstage)	Indireta	Excelente	Indireta

Tabela 1 – Quando usar cada gerador
Fonte: Elaborado pelo autor (2025)

Arquitetura de build, performance e recursos

Geradores estáticos percorrem N nós (arquivos + símbolos). De modo simplificado, o tempo de build é $T \approx O(N)$ para parsing e render, com passos que podem crescer por seções de exemplos executados (e.g., sphinx-gallery). Para times com muitos módulos, adote caching de ambiente, pin de versões, mocks de dependências pesadas (autodoc_mock_imports) e builds incrementais em CI. A hospedagem estática (GitHub Pages, Read the Docs) mantém o custo computacional fora do servidor de produção.

PDF exclusivo para Julia Soares Pereira - rm373605
jusoper@gmail.com

Memória/CPU: autodoc importa o pacote e se houver efeitos colaterais em import, eles serão executados — isole side effects atrás de if __name__ == "__main__":, use feature flags e mocks para dependências nativas pesadas. Em pipelines compartilhados, limite workers do builder ou off-load de notebooks (nbval) para etapas separadas.

Qualidade contínua: métricas e testes de documentação

- **Cobertura de docstrings**: interrogate e docstr-coverage calculam coverage por módulo/classe/função; falham o build abaixo de um threshold (ex.: 90%). Integre em pre-commit e no CI.
- **Executável e reprodutível**: doctest + pytest --doctest-modules + nbval para notebooks. Use sanitizers (regex) quando saídas variam (timestamps).
- **Estilo**: PEP 257 + padrão NumPy (ou Google) harmonizam seções e reduzem entropia textual. Cite type hints para reforçar contratos e gerar assinaturas precisas nos docs.

Para portais internos, o Backstage/TechDocs adota MkDocs sob o capô: Markdown versionado junto ao código, prepare → generate → publish e leitura no portal (com suporte a S3/GCS/Blob). Isso se encaixa no fluxo de SDKs internos e centraliza discoverability.

FIAP

PDF exclusivo para Julia Soares Pereira - rm373605
jusoper@gmail.com

MERCADO, CASES E TENDÊNCIAS

Casos de referência (boas práticas públicas)

A **Stripe API Reference** é frequentemente citada como padrão ouro de referência factual (busca, exemplos de requisição/resposta, SDKs e OpenAPI). Observe como a referência é separada de guias, aderindo ao espírito Diátaxis.

O **Twilio Docs** também investe em consistência cross-API (REST, SDKs, OpenAPI, quickstarts) — é útil para comparar estratégias de "multiproduto/um só estilo".

O **FastAPI** exibe documentação interativa automática (OpenAPI/Swagger, ReDoc), ilustrando o valor de tipos e exemplos para reduzir bugs e acelerar o onboarding.

DevPortal interno (docs-as-code)

O **Backstage TechDocs** (Spotify) tornou-se de fato para portais internos: Markdown na raiz, mkdocs.yml, geração local/CI e publicação em storage (S3/GCS). A clareza de papéis (preparer → generator → publisher) reduz o atrito operacional.

Falhas recorrentes (anti-cases)

- Misturar how-to com referência (o leitor "trava").
- Doc gerada que diverge do código por falta de doctest/CI.
- API sem exemplos mínimos executáveis.
- Doc "bonita" sem versão (sem changelog e SemVer) — o usuário não sabe se quebrou.

Mitigações: Diátaxis + testes executáveis + versionamento claro (**SemVer**, Keep a Changelog).

FIAP

PDF exclusivo para Julia Soares Pereira - rm373605
jusoper@gmail.com

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, vimos por que a documentação é um ativo crítico para bibliotecas internas/SDKs, além de explorar docstrings bem formadas (PEP 257) + type hints (PEP 484) como base para autodocumentação.

Também estudamos como Sphinx + napoleon e MkDocs geram sites a partir do código, como testar/medir a documentação (doctest, pytest, nbval, interrogate) e, por fim, como estruturar docs com Diátaxis e docs-as-code (TechDocs/Read the Docs).

FIAP

PDF exclusivo para Julia Soares Pereira - rm373605
jusoper@gmail.com

REFERÊNCIAS

BACKSTAGE.IO. **TechDocs (Backstage)**. 2025. Disponível em: https://backstage.io/docs/features/techdocs/ . Acesso em: 19 jan. 2026.

DIATAXIS.FR. **Diátaxis — Framework de documentação**. 2026. Disponível em: https://diataxis.fr/ . Acesso em: 19 jan. 2026.

DOCS.STRIPE.COM. **Stripe — API Reference**. 2026. Disponível em: https://docs.stripe.com/api . Acesso em: 19 jan. 2026.

DOCUTILS. **PEP 257 — Docstring Conventions**. 29 mai. 2001. Disponível em: https://peps.python.org/pep-0257/ . Acesso em: 19 jan. 2026.

MKDOCS.ORG. **MkDocs — Site oficial**. 2026. Disponível em: https://www.mkdocs.org/ . Acesso em: 19 jan. 2026.

NBVAL.READTHEDOCS.IO. **nbval — Pytest plugin para notebooks**. 2026. Disponível em: https://nbval.readthedocs.io/en/latest/ . Acesso em: 19 jan. 2026.

NUMPYDOC.READTHEDOCS.IO. **NumPyDoc Style Guide**. 2026. Disponível em: https://numpydoc.readthedocs.io/en/latest/format.html . Acesso em: 19 jan. 2026.

READTHEDOCS.COM. **Read the Docs — Sphinx**. 2026. Disponível em: https://docs.readthedocs.com/platform/stable/intro/sphinx.html . Acesso em: 19 jan. 2026.

READTHEDOCS.IO. **interrogate — Docstring coverage**. 2026. Disponível em: https://interrogate.readthedocs.io/en/latest/ . Acesso em: 19 jan. 2026.

SPHINXCONTRIB-NAPOLEON.READTHEDOCS.IO. **Sphinx — napoleon (Google/NumPy style)**. 2026. Disponível em: https://sphinxcontrib-napoleon.readthedocs.io/en/latest/ . Acesso em: 19 jan. 2026.

SPHINX-DOC.ORG. **Sphinx — autodoc**. 2026. Disponível em: https://www.sphinx-doc.org/en/master/usage/extensions/autodoc.html . Acesso em: 19 jan. 2026.

SPHINX-GALLERY.GITHUB.IO. **Sphinx Gallery — documentação**. 2026. Disponível em: https://sphinx-gallery.github.io/stable/index.html . Acesso em: 19 jan. 2026.

TWILIO DOCS. **Twilio Docs**. 2026. Disponível em: https://www.twilio.com/docs. Acesso em: 19 jan. 2026.

PDF exclusivo para Julia Soares Pereira - rm373605
jusoper@gmail.com

TWILIO.COM. **Twilio — Docs**. 2026. Disponível em: https://www.twilio.com/docs . Acesso em: 19 jan. 2026.

FIAP

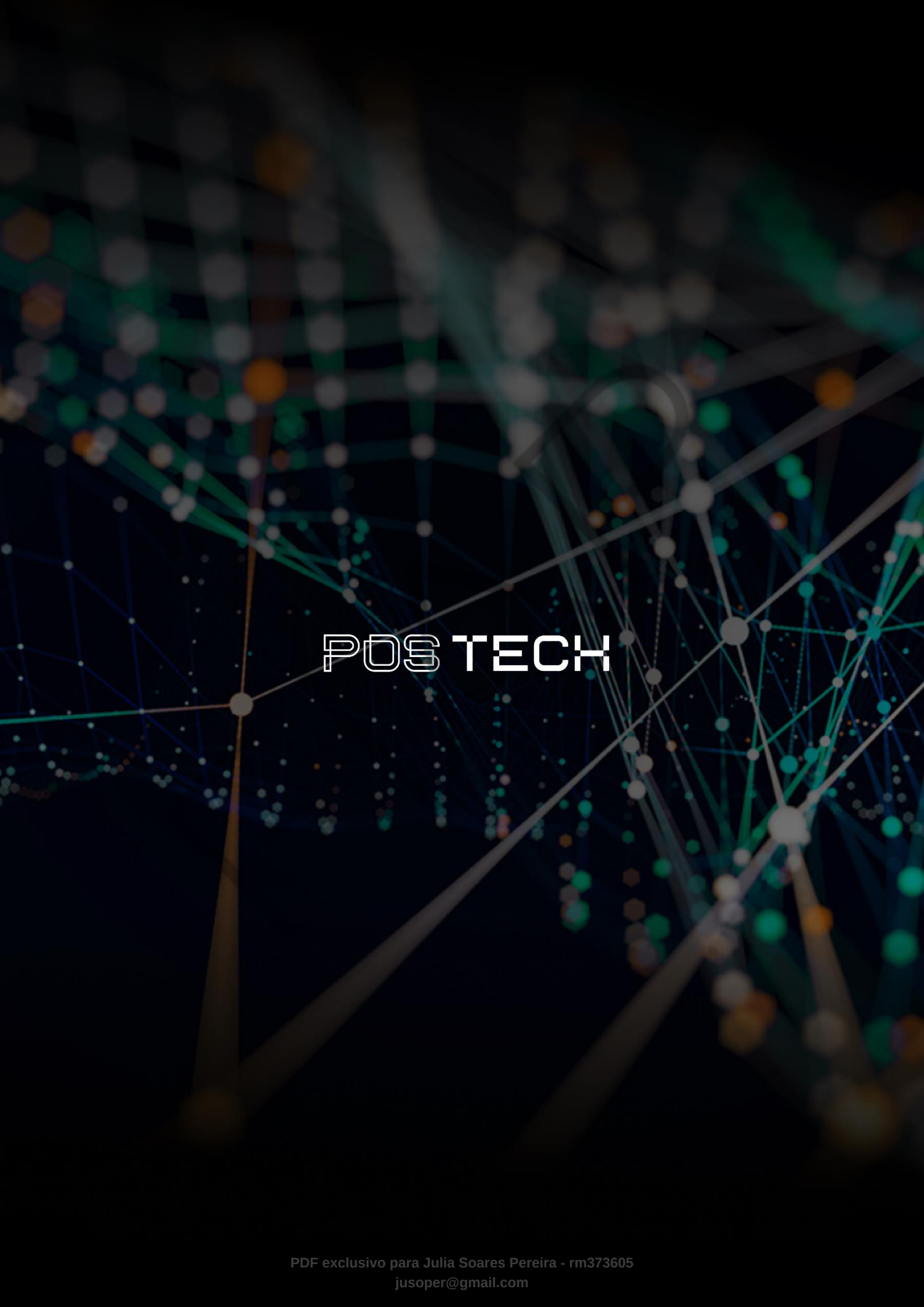
PDF exclusivo para Julia Soares Pereira - rm373605
jusoper@gmail.com

PALAVRAS-CHAVE

docs-as-code. docstrings (PEP 257/NumPy/Google). Sphinx/MkDocs (autodoc/napoleon).

FIAP

PDF exclusivo para Julia Soares Pereira - rm373605
jusoper@gmail.com



PDF exclusivo para Julia Soares Pereira - rm373605
jusoper@gmail.com